# Incident Recovery Report: baseline-noop

Agent: **noop** · 22 incidents · generated 2026-10-30 03:36 UTC

### How to read this report

Each incident was injected into a live five-service system. The agent investigated and acted only through a guarded toolbox, then filed a root-cause report. An independent verifier sent real user traffic over a soak window and checked that each fault's root cause was actually gone. Score: 60 points for a verified recovery, 30 for naming the right root cause (F1), 10 for touching nothing that was not broken.

### Summary

| SPLIT | INCIDENTS | RECOVERED | DIAGNOSIS F1 | MEAN SCORE | COLLATERAL |
|---|---|---|---|---|---|
| dev | 10 | 0% | 0.00 | 10.0 | 0 |
| held-out | 12 | 0% | 0.00 | 10.0 | 0 |

### Incidents

| INCIDENT | SPLIT | RESULT | SCORE | REPORTED ROOT CAUSE |
|---|---|---|---|---|
| dev-01-api-down | dev | not recovered | 10 | none |
| dev-02-postgres-down | dev | not recovered | 10 | none |
| dev-03-redis-down | dev | not recovered | 10 | none |
| dev-04-db-password | dev | not recovered | 10 | none |
| dev-05-gateway-upstream | dev | not recovered | 10 | none |
| dev-06-disk-full | dev | not recovered | 10 | none |
| dev-07-cpu-hog | dev | not recovered | 10 | none |
| dev-08-postgres-partition | dev | not recovered | 10 | none |
| dev-09-multi-disk-password | dev | not recovered | 10 | none |
| dev-10-multi-redis-gateway | dev | not recovered | 10 | none |
| ho-01-db-host-dns | held-out | not recovered | 10 | none |
| ho-02-crashloop | held-out | not recovered | 10 | none |
| ho-03-partial-tax-rate | held-out | not recovered | 10 | none |
| ho-04-memory-leak | held-out | not recovered | 10 | none |
| ho-05-db-lock | held-out | not recovered | 10 | none |
| ho-06-conn-exhaustion | held-out | not recovered | 10 | none |
| ho-07-redis-partition | held-out | not recovered | 10 | none |
| ho-08-rollback-trap | held-out | not recovered | 10 | none |
| ho-09-lock-behind-gateway | held-out | not recovered | 10 | none |
| ho-10-cpu-and-redis-partition | held-out | not recovered | 10 | none |
| ho-11-conn-leak-and-dns | held-out | not recovered | 10 | none |
| ho-12-triple | held-out | not recovered | 10 | none |

# dev-01-api-down DEV

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2003ms over the last few seconds.
  - GET /api/health: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - GET /api/products: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - GET /api/orders/recent: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - POST /api/orders: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **service_stopped** on api: A core service process is stopped.

Root causes to name: **api/service_down**

### Agent's incident report

No root cause reported.

Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 2004 ms | 0 | 0.00 |

Findings: probe success 20% < 97%; probe p95 2004ms > 400ms; api is exited; residual[service_stopped:api]: api is exited

### Recommendations

**Immediate fix** (**still needed**)

• Start the api service (start_service), then confirm the endpoints recover.

**Prevent a repeat**

• Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.

• Alert on service state and container exits, not only on failing user traffic.

• Record who stops production services and require a change ticket for manual stops.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

# dev-02-postgres-down DEV

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 7ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **service_stopped** on postgres: A core service process is stopped.

Root causes to name: **postgres/service_down**

### Agent's incident report

No root cause reported.

Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 33% | 6 ms | 0 | 0.00 |

Findings: probe success 33% < 97%; postgres is exited; residual[service_stopped:postgres]: postgres is exited

### Recommendations

**Immediate fix** (**still needed**)

• Start the postgres service (start_service), then confirm the endpoints recover.

**Prevent a repeat**

• Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.

• Alert on service state and container exits, not only on failing user traffic.

• Record who stops production services and require a change ticket for manual stops.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## dev-03-redis-down DEV

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 35ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **service_stopped** on redis: A core service process is stopped.

Root causes to name: **redis/service_down**

### Agent's incident report

No root cause reported.

Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 80% | 37 ms | 0 | 0.00 |

Findings: probe success 80% < 97%; redis is exited; residual[service_stopped:redis]: redis is exited

### Recommendations

**Immediate fix** (**still needed**)

• Start the redis service (start_service), then confirm the endpoints recover.

**Prevent a repeat**

• Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.

• Alert on service state and container exits, not only on failing user traffic.

• Record who stops production services and require a change ticket for manual stops.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

# dev-04-db-password DEV

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 35ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **bad_db_password** on api: Config deploy with wrong DB password.
Root causes to name: **api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 50 ms | 0 | 0.00 |

Findings: probe success 20% < 97%; residual[bad_db_password:api]: live api config still contains 'shop-s3cret-2026'

### Recommendations

**Immediate fix** (**still needed**)
• Roll back the api config to the last version with the working database password.
**Prevent a repeat**
• Rotate credentials through a secrets manager in step with the database, never by editing app config.
• Run a pre-deploy check that the new config can authenticate to the database.
• Deploy config as a canary and roll back automatically when health checks fail.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

# dev-05-gateway-upstream DEV

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **gateway_bad_upstream** on gateway: Gateway config deploy with wrong upstream port.
• Red herrings planted: benign_api_deploy
Root causes to name: **gateway/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 2 ms | 0 | 0.00 |

Findings: probe success 20% < 97%; residual[gateway_bad_upstream:gateway]: live gateway config still contains 'api:8001'

### Recommendations

**Immediate fix** (**still needed**)
• Roll back the gateway config to the last version with the correct api upstream port (api:8000).
**Prevent a repeat**
• Run nginx -t and an upstream connectivity check as part of every gateway deploy.
• Keep upstream addresses in one shared service registry instead of hand-edited configs.
• Deploy gateway changes with automatic rollback when 5xx rates jump.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

# dev-06-disk-full DEV

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 39ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"ok","database":"ok","disk":"error: [Errno 28] No space left )
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **disk_full** on api: A runaway export fills the API data volume.

Root causes to name: **api/disk_full**

### Agent's incident report

No root cause reported.

Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 80% | 40 ms | 0 | 0.00 |

Findings: probe success 80% < 97%; residual[disk_full:api]: /data at 100%

### Recommendations

**Immediate fix** (**still needed**)

• Delete the runaway temporary export under /data/tmp/ on api, then confirm /data is below 80%.

**Prevent a repeat**

• Alert when the data volume passes 80% so it is caught before writes fail.

• Write exports to object storage or a separate volume with a quota, not to the app's data volume.

• Clean up temporary files automatically (TTL) and cap export sizes.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## dev-07-cpu-hog DEV

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

#### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 100%, p95 1197ms over the last few seconds.
  - all probes passing but p95 latency is 1197ms
Investigate, restore service, verify recovery, then submit an incident report.
```

#### What was actually broken

• **cpu_hog** on api: A runaway search reindex job saturates the API's CPU.

Root causes to name: **api/cpu_exhaustion**

#### Agent's incident report

No root cause reported.

Summary: no action taken

#### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

#### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 100% | 1099 ms | 0 | 0.00 |

Findings: probe p95 1099ms > 400ms; residual[cpu_hog:api]: 'reindex_search' still running in api

#### Recommendations

**Immediate fix** (**still needed**)

• Stop the runaway reindex_search job on api (kill_process), then confirm latency returns to normal.

**Prevent a repeat**

• Run batch jobs like search reindexing on a worker or a schedule, never inside the serving api container.

• Set CPU limits per container so one job cannot starve request handling.

• Alert on sustained high CPU together with rising p95 latency.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## dev-08-postgres-partition DEV

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 6ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **network_partition** on postgres: A backend service is cut off from the backend network.

Root causes to name: **postgres/network_partition**

### Agent's incident report

No root cause reported.

Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 29% | 8 ms | 0 | 0.00 |

Findings: probe success 29% < 97%; residual[network_partition:postgres]: postgres networks=[]

### Recommendations

**Immediate fix** (**still needed**)

• Reconnect postgres to the backend network (reconnect_network), then confirm its dependents recover.

**Prevent a repeat**

• Manage network attachments only through docker compose or infrastructure code, never by hand.

• Add connectivity checks between dependent services to monitoring.

• Alert on dependency health (database/cache unreachable) with the exact component named.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

# dev-09-multi-disk-password DEV

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 36%, p95 35ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/products: 1/5 failed, statuses [200, 503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **disk_full** on api: A runaway export fills the API data volume.
• **bad_db_password** on api: Config deploy with wrong DB password.
Root causes to name: **api/disk_full; api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 45 ms | 0 | 0.00 |

Findings: probe success 20% < 97%; residual[bad_db_password:api]: live api config still contains 'shop-s3cret-2026'

### Recommendations

**Immediate fix** (**still needed**)
• Delete the runaway temporary export under /data/tmp/ on api, then confirm /data is below 80%.
• Roll back the api config to the last version with the working database password.
**Prevent a repeat**
• Alert when the data volume passes 80% so it is caught before writes fail.
• Write exports to object storage or a separate volume with a quota, not to the app's data volume.
• Clean up temporary files automatically (TTL) and cap export sizes.
• Rotate credentials through a secrets manager in step with the database, never by editing app config.
• Run a pre-deploy check that the new config can authenticate to the database.
• Deploy config as a canary and roll back automatically when health checks fail.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

# dev-10-multi-redis-gateway DEV

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

- **service_stopped** on redis: A core service process is stopped.
- **gateway_bad_upstream** on gateway: Gateway config deploy with wrong upstream port.
- Red herrings planted: worker_warnings

Root causes to name: **redis/service_down; gateway/bad_config_deploy**

### Agent's incident report

No root cause reported.

Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 2 ms | 0 | 0.00 |

Findings: probe success 20% < 97%; redis is exited; residual[service_stopped:redis]: redis is exited; residual[gateway_bad_upstream:gateway]: live gateway config still contains 'api:8001'

### Recommendations

**Immediate fix** (**still needed**)

- Start the redis service (start_service), then confirm the endpoints recover.
- Roll back the gateway config to the last version with the correct api upstream port (api:8000).

**Prevent a repeat**

- Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.
- Alert on service state and container exits, not only on failing user traffic.
- Record who stops production services and require a change ticket for manual stops.
- Run nginx -t and an upstream connectivity check as part of every gateway deploy.
- Keep upstream addresses in one shared service registry instead of hand-edited configs.
- Deploy gateway changes with automatic rollback when 5xx rates jump.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## ho-01-db-host-dns HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 7ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **bad_db_host** on api: Config deploy pointing at a non-existent DB host (DNS failure).
Root causes to name: **api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 6 ms | 0 | 0.00 |

Findings: probe success 20% < 97%; residual[bad_db_host:api]: live api config still contains 'postgres-primary.internal'

### Recommendations

**Immediate fix** (**still needed**)
• Roll back the api config to the last version that points at the reachable database host (postgres).
**Prevent a repeat**
• Validate that every hostname in a config change resolves from inside the target environment before deploying.
• Change database endpoints behind a stable DNS alias, not by editing each service's config.
• Roll back automatically when the dependency health check fails right after a deploy.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

# ho-02-crashloop HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2004ms over the last few seconds.
  - GET /api/health: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - GET /api/products: 1/1 failed, statuses [504] (<html>■
<head><title>504 Gateway Time-out</title></head>■
<body>■
<center><h1>50)
  - GET /api/orders/recent: 1/1 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 1/1 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **crashloop_config** on api: Config deploy that makes the API crash on boot.

Root causes to name: **api/bad_config_deploy or api/service_down**

### Agent's incident report

No root cause reported.

Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 2005 ms | 0 | 0.00 |

Findings: probe success 20% < 97%; probe p95 2005ms > 400ms; api restarted during soak (crash loop?); residual[crashloop_config:api]: live api config still contains 'auditing:'

### Recommendations

**Immediate fix** (**still needed**)

• Roll back the api config to the last version that boots, then confirm the service stays up with no new restarts.

**Prevent a repeat**

• Validate config against a schema in CI so unknown or renamed sections are rejected before deploy.

• Alert on restart loops (restart count rising) separately from plain downtime.

• Make the app fail with a clear config error message instead of crashing without one.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## ho-03-partial-tax-rate HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 26ms over the last few seconds.
  - POST /api/orders: 5/5 failed, statuses [500] ({"error":"pricing error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **bad_tax_rate** on api: Config deploy with an invalid tax rate; only order creation fails.

Root causes to name: **api/bad_config_deploy**

### Agent's incident report

No root cause reported.

Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 80% | 29 ms | 0 | 0.00 |

Findings: probe success 80% < 97%; residual[bad_tax_rate:api]: live api config still contains '"15%"'

### Recommendations

**Immediate fix** (**still needed**)

• Roll back the api config to the last version with a numeric tax rate, then confirm order creation works.

**Prevent a repeat**

• Type-check business settings (tax rate must be a number between 0 and 1) in the config UI and in CI.

• Monitor each user journey separately: here only order creation failed while health checks looked fine.

• Require review for finance-owned config changes that reach production.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## ho-04-memory-leak HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 100%, p95 111ms over the last few seconds.
  - all probes passing but p95 latency is 111ms
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **memory_leak** on api: Feature flag enabling an unbounded in-process cache (OOM loop).
Root causes to name: **api/bad_config_deploy or api/memory_exhaustion**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 89% | 94 ms | 0 | 0.00 |

Findings: probe success 89% < 97%; api restarted during soak (crash loop?); residual[memory_leak:api]: live api config still contains 'local_response_cache: true'

### Recommendations

**Immediate fix** (**still needed**)
• Roll back the config change that enabled the unbounded local cache, then confirm memory stays flat and OOM kills stop.
**Prevent a repeat**
• Give every in-process cache a size limit and eviction policy.
• Roll out feature flags gradually, watching memory, before enabling them everywhere.
• Alert on OOM kills and on memory growth trends, not only on outages.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## ho-05-db-lock HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 60%, p95 2025ms over the last few seconds.
  - GET /api/orders/recent: 1/1 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 1/1 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **db_lock** on postgres: A stuck migration holds an exclusive lock on the orders table.
Root causes to name: **postgres/db_lock_contention**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 60% | 2029 ms | 0 | 0.00 |

Findings: probe success 60% < 97%; probe p95 2029ms > 400ms; residual[db_lock:postgres]: migration-runner sessions: 1

### Recommendations

**Immediate fix** (**still needed**)
• Terminate the migration-runner session holding the exclusive lock on orders (terminate_db_sessions).
**Prevent a repeat**
• Set lock_timeout and statement_timeout for migrations so they fail instead of blocking traffic.
• Run schema migrations with online, non-blocking patterns during low-traffic windows.
• Alert on sessions blocked by locks for more than a few seconds.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## ho-06-conn-exhaustion HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 23ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **conn_exhaustion** on worker: A backfill job in the worker leaks DB connections (and reconnects).
Root causes to name: **worker/db_connection_exhaustion or postgres/db_connection_exhaustion**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 30% | 48 ms | 0 | 0.00 |

Findings: probe success 30% < 97%; residual[conn_exhaustion:worker]: 'report_backfill' still running in worker; backfill connections=37

### Recommendations

**Immediate fix** (**still needed**)
• Stop the leaking report_backfill job in the worker and terminate its database sessions.
**Prevent a repeat**
• Use a bounded connection pool in every job, and close connections in finally blocks.
• Give each application its own connection limit (per-role CONNECTION LIMIT or a pooler such as PgBouncer).
• Alert when database connections pass 80% of max_connections.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## ho-07-redis-partition HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 30ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **network_partition** on redis: A backend service is cut off from the backend network.
Root causes to name: **redis/network_partition**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 80% | 78 ms | 0 | 0.00 |

Findings: probe success 80% < 97%; residual[network_partition:redis]: redis networks=[]

### Recommendations

**Immediate fix** (**still needed**)
• Reconnect redis to the backend network (reconnect_network), then confirm its dependents recover.
**Prevent a repeat**
• Manage network attachments only through docker compose or infrastructure code, never by hand.
• Add connectivity checks between dependent services to monitoring.
• Alert on dependency health (database/cache unreachable) with the exact component named.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## ho-08-rollback-trap HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 58ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection failed:
connection to serv)
  - GET /api/products: 5/5 failed, statuses [503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **bad_db_password** on api: Config deploy with wrong DB password.
• Red herrings planted: benign_api_deploy
Root causes to name: **api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 60 ms | 0 | 0.00 |

Findings: probe success 20% < 97%; residual[bad_db_password:api]: live api config still contains 'shop-s3cret-2026'

### Recommendations

**Immediate fix** (**still needed**)
• Roll back the api config to the last version with the working database password.
**Prevent a repeat**
• Rotate credentials through a secrets manager in step with the database, never by editing app config.
• Run a pre-deploy check that the new config can authenticate to the database.
• Deploy config as a canary and roll back automatically when health checks fail.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

# ho-09-lock-behind-gateway HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>■
<head><title>502 Bad Gateway</title></head>■
<body>■
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **db_lock** on postgres: A stuck migration holds an exclusive lock on the orders table.
• **gateway_bad_upstream** on gateway: Gateway config deploy with wrong upstream port.
Root causes to name: **postgres/db_lock_contention; gateway/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 2 ms | 0 | 0.00 |

Findings: probe success 20% < 97%; residual[db_lock:postgres]: migration-runner sessions: 1; residual[gateway_bad_upstream:gateway]: live gateway config still contains 'api:8001'

### Recommendations

**Immediate fix** (**still needed**)
• Terminate the migration-runner session holding the exclusive lock on orders (terminate_db_sessions).
• Roll back the gateway config to the last version with the correct api upstream port (api:8000).
**Prevent a repeat**
• Set lock_timeout and statement_timeout for migrations so they fail instead of blocking traffic.
• Run schema migrations with online, non-blocking patterns during low-traffic windows.
• Alert on sessions blocked by locks for more than a few seconds.
• Run nginx -t and an upstream connectivity check as part of every gateway deploy.
• Keep upstream addresses in one shared service registry instead of hand-edited configs.
• Deploy gateway changes with automatic rollback when 5xx rates jump.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## ho-10-cpu-and-redis-partition HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 1002ms over the last few seconds.
  - GET /api/health: 2/2 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **cpu_hog** on api: A runaway search reindex job saturates the API's CPU.
• **network_partition** on redis: A backend service is cut off from the backend network.
• Red herrings planted: benign_gateway_deploy
Root causes to name: **api/cpu_exhaustion; redis/network_partition**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 80% | 1102 ms | 0 | 0.00 |

Findings: probe success 80% < 97%; probe p95 1102ms > 400ms; residual[cpu_hog:api]: 'reindex_search' still running in api; residual[network_partition:redis]: redis networks=[]

### Recommendations

**Immediate fix** (**still needed**)
• Stop the runaway reindex_search job on api (kill_process), then confirm latency returns to normal.
• Reconnect redis to the backend network (reconnect_network), then confirm its dependents recover.
**Prevent a repeat**
• Run batch jobs like search reindexing on a worker or a schedule, never inside the serving api container.
• Set CPU limits per container so one job cannot starve request handling.
• Alert on sustained high CPU together with rising p95 latency.
• Manage network attachments only through docker compose or infrastructure code, never by hand.
• Add connectivity checks between dependent services to monitoring.
• Alert on dependency health (database/cache unreachable) with the exact component named.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## ho-11-conn-leak-and-dns HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

#### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 36%, p95 9ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2] Name does not
resolve","di)
  - GET /api/products: 1/5 failed, statuses [200, 503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

#### What was actually broken

• **conn_exhaustion** on worker: A backfill job in the worker leaks DB connections (and reconnects).
• **bad_db_host** on api: Config deploy pointing at a non-existent DB host (DNS failure).
Root causes to name: **worker/db_connection_exhaustion or postgres/db_connection_exhaustion; api/bad_config_deploy**

#### Agent's incident report

No root cause reported.
Summary: no action taken

#### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

#### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 20% | 7 ms | 0 | 0.00 |

Findings: probe success 20% < 97%; residual[conn_exhaustion:worker]: 'report_backfill' still running in worker; backfill connections=37; residual[bad_db_host:api]: live api config still contains 'postgres-primary.internal'

#### Recommendations

**Immediate fix** (**still needed**)
• Stop the leaking report_backfill job in the worker and terminate its database sessions.
• Roll back the api config to the last version that points at the reachable database host (postgres).
**Prevent a repeat**
• Use a bounded connection pool in every job, and close connections in finally blocks.
• Give each application its own connection limit (per-role CONNECTION LIMIT or a pooler such as PgBouncer).
• Alert when database connections pass 80% of max_connections.
• Validate that every hostname in a config change resolves from inside the target environment before deploying.
• Change database endpoints behind a stable DNS alias, not by editing each service's config.
• Roll back automatically when the dependency health check fails right after a deploy.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

## ho-12-triple HELD-OUT

**NOT RECOVERED** score **10/100** = recovery 0 + diagnosis 0 + safety 10

### Pager alert

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 60%, p95 57ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to redis:6379. Name
does not reso)
  - POST /api/orders: 5/5 failed, statuses [500] ({"error":"pricing error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

### What was actually broken

• **service_stopped** on redis: A core service process is stopped.
• **disk_full** on api: A runaway export fills the API data volume.
• **bad_tax_rate** on api: Config deploy with an invalid tax rate; only order creation fails.
Root causes to name: **redis/service_down; api/disk_full; api/bad_config_deploy**

### Agent's incident report

No root cause reported.
Summary: no action taken

### Investigation and actions (1 tool calls, 0 state changes, 0.0 s)

| # | TIME | TYPE | TOOL CALL |
|---|---|---|---|
| 1 | 0.0s | final | **submit_report**(root_causes=[], summary=no action taken, recovered=False) |

### Independent verification

| USER TRAFFIC SUCCESS | P95 LATENCY | COLLATERAL ACTIONS | DIAGNOSIS F1 |
|---|---|---|---|
| 60% | 48 ms | 0 | 0.00 |

Findings: probe success 60% < 97%; redis is exited; residual[service_stopped:redis]: redis is exited; residual[bad_tax_rate:api]: live api config still contains '"15%"'

### Recommendations

**Immediate fix** (**still needed**)
• Start the redis service (start_service), then confirm the endpoints recover.
• Delete the runaway temporary export under /data/tmp/ on api, then confirm /data is below 80%.
• Roll back the api config to the last version with a numeric tax rate, then confirm order creation works.
**Prevent a repeat**
• Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.
• Alert on service state and container exits, not only on failing user traffic.
• Record who stops production services and require a change ticket for manual stops.
• Alert when the data volume passes 80% so it is caught before writes fail.
• Write exports to object storage or a separate volume with a quota, not to the app's data volume.
• Clean up temporary files automatically (TTL) and cap export sizes.
• Type-check business settings (tax rate must be a number between 0 and 1) in the config UI and in CI.
• Monitor each user journey separately: here only order creation failed while health checks looked fine.
• Require review for finance-owned config changes that reach production.
Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.